

Building a Fully Automated XDR/SOAR Detection & Response Pipeline

Wazuh SIEM · Shuffle SOAR · TheHive · Cortex · OPNsense

A Home Security Operations Center (SOC) Lab Project
From Detection to Automated Containment – End-to-End SOAR Engineering

Author: Mehenni May Fatma Zohra
Role: Cybersecurity Engineering Student
Institution: USTHB, Algiers
GitHub: github.com/mayversion/soc-analyst-portfolio
Date: August 2026

Contents

1	Executive Summary	2
2	Introduction and Objectives	2
3	Lab Architecture	3
3.1	Network Segmentation	3
3.2	Component Overview	4
4	Migration Rationale: From Splunk to an Open-Source XDR/SOAR Stack	4
5	Infrastructure Deployment	5
5.1	Wazuh Manager and Agent Fleet	5
5.2	Log Source Coverage	6
6	Automated Response Pipeline Design	9
6.1	Pipeline Flow	9
6.2	Active Response Configuration	10
6.3	Debugging Journey: the <location> Dispatch Bug	11
6.4	Attacker Interaction Capture	12
7	Shuffle SOAR Workflow	13
7.1	Workflow Node Breakdown	14
8	TheHive Case Management	17
9	Cortex Threat Intelligence Enrichment	20
10	OPNsense Automated Containment	21
11	End-to-End Validation and Test Scenarios	22
11.1	Proof of End-to-End Execution	22
11.2	Scenario 1: SSH Brute-Force Detection	23
11.3	Scenario 2: Cowrie Honeypot Interaction	25
12	Threat Hunting and Monitoring Dashboards	26
13	Challenges and Lessons Learned	27
14	Conclusion and Future Work	28

1 Executive Summary

This report documents the design, deployment, and validation of a fully automated Extended Detection and Response (XDR) / Security Orchestration, Automation and Response (SOAR) pipeline built entirely within a home lab environment. The project evolved from an earlier Splunk-based SOC lab into a modern, open-source detection and response stack composed of **Wazuh** (SIEM / XDR agent manager), **Shuffle** (SOAR orchestrator), **TheHive** (case management), **Cortex** (threat intelligence enrichment), and **OPNsense** (perimeter firewall used as an active response target).

The core objective was to build a pipeline capable of detecting a real attack (an SSH brute-force attempt against an internal Cowrie honeypot), triggering an automated Active Response on the Wazuh manager, orchestrating a full incident response workflow through Shuffle, enriching the offending IP address with threat intelligence via Cortex analyzers, automatically opening and populating a case in TheHive, and finally pushing a firewall block rule to OPNsense – all without human intervention. The project also includes several rounds of hands-on debugging of the Wazuh Active Response dispatch mechanism, which forms a significant part of the engineering narrative below.

Key Outcomes

- Migrated a 5-zone virtualized SOC lab from Splunk Enterprise to a Wazuh + Shuffle + Cortex + TheHive stack.
- Resolved a critical `<location>local</location>` vs. `<location>server</location>` Active Response dispatch bug that silently prevented webhook execution.
- Achieved end-to-end automation: detection → webhook → case creation → enrichment → containment → case resolution.
- Validated the pipeline against two live attack scenarios: an SSH brute-force against a monitored host, and a successful honeypot login with command execution captured by Cowrie.

2 Introduction and Objectives

Security teams increasingly rely on SOAR platforms to reduce Mean Time to Respond (MTTR) by automating the repetitive triage and containment steps that would otherwise consume analyst time. As part of my cybersecurity engineering training and my ongoing SOC/Blue Team specialization, I wanted to move beyond passive log collection and build a lab where a detection could *automatically* trigger a complete incident response lifecycle: alerting, enrichment, case documentation, and containment.

The objectives of this project were to:

1. Replace the previous Splunk-centric SOC lab with an open-source, resource-light XDR/SOAR stack better suited to a home lab environment.
2. Design a segmented network architecture reflecting real enterprise zoning practices (LAN, DMZ, Honeypot, Management).
3. Configure Wazuh Active Response to fire a webhook toward a SOAR orchestrator whenever a high-severity rule is triggered.
4. Build a Shuffle workflow that creates a case in TheHive, enriches observables through Cortex analyzers (VirusTotal, AbuseIPDB, OTX), and pushes a containment action to the firewall.

- Validate the entire pipeline against realistic attack scenarios and document the debugging process.

3 Lab Architecture

3.1 Network Segmentation

The lab is built around five isolated network zones, all routed through an OPNsense firewall acting as the central gateway. East-west traffic between zones is explicitly restricted by firewall rules, and management access is only permitted from the dedicated MGMT zone.

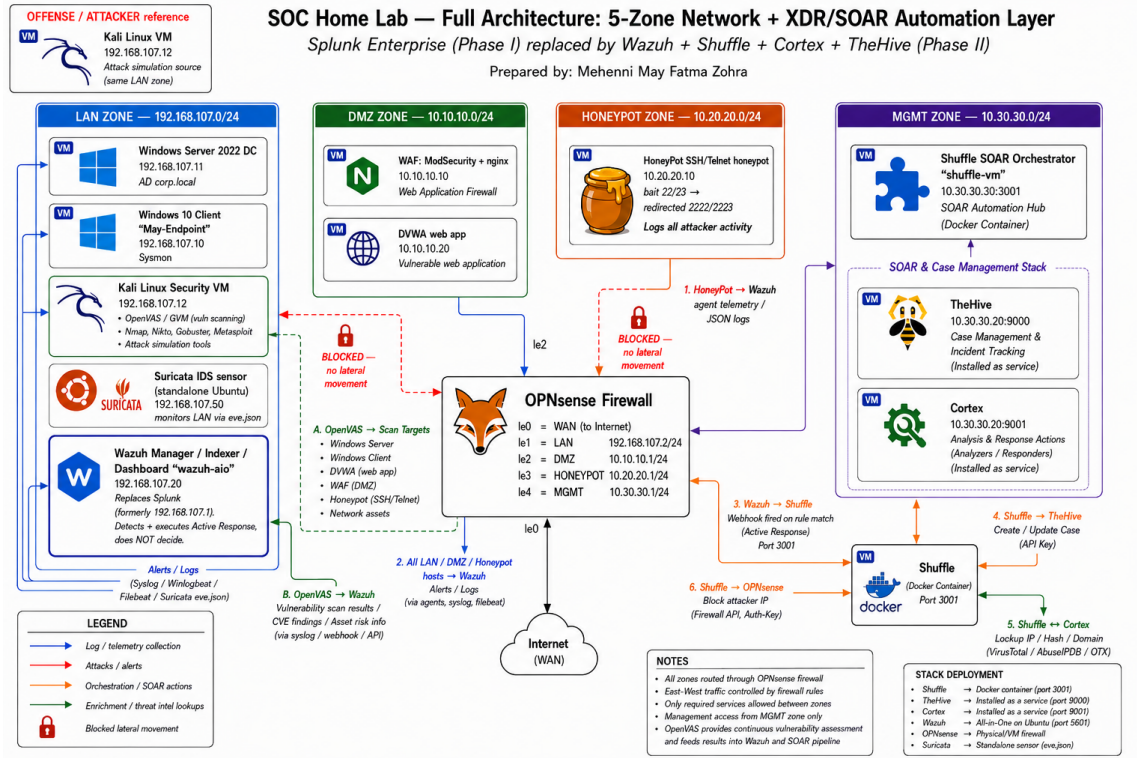


Figure 1: Full SOC home lab architecture: 5-zone network with the XDR/SOAR automation layer overlaid on top of the legacy Splunk (Phase I) topology, now replaced by Wazuh + Shuffle + Cortex + TheHive (Phase II).

Table 1: Network zones and hosted assets

Zone	Subnet	Key Assets
LAN	192.168.107.0/24	Windows Server 2022 DC, Windows 10 client, Kali attacker, Suricata IDS, Wazuh manager
DMZ	10.10.10.0/24	NGINX + ModSecurity WAF, DVWA vulnerable web app
Honeypot	10.20.20.0/24	Cowrie SSH/Telnet honeypot (bait ports 22/23 → 2222/2223)
MGMT	10.30.30.0/24	Shuffle SOAR (Docker), TheHive and Cortex (installed as native services)
Perimeter	–	OPNsense firewall (all interfaces / inter-zone routing)

3.2 Component Overview

Table 2: Core stack components and their role

Component	Role
Wazuh Manager/Indexer/Dashboard	Central log collection, rule correlation, and Active Response execution. Detects; does not decide on final containment.
Shuffle	SOAR orchestrator. Receives Wazuh webhooks and drives the automated response workflow.
TheHive	Case management platform; stores incidents, observables, and analyst tasks.
Cortex	Threat intelligence enrichment engine (VirusTotal, AbuseIPDB, AlienVault OTX analyzers).
OPNsense	Perimeter firewall; final containment target for automated IP blocking.
Cowrie	Medium-interaction SSH/Telnet honeypot used to generate realistic attacker telemetry.
Suricata	Network IDS monitoring LAN traffic.

4 Migration Rationale: From Splunk to an Open-Source XDR/-SOAR Stack

The lab originally ran on Splunk Enterprise for log aggregation, with manual review of alerts. While functional for log search and dashboarding, Splunk’s resource footprint and licensing model made it a poor fit for continuous home-lab operation, and it offered no native SOAR capability. The migration to Wazuh (SIEM/XDR) paired with Shuffle (SOAR), TheHive (case management), and Cortex (enrichment) was driven by three factors:

- **Cost and resource efficiency** – all four platforms are open-source and run comfortably on a single MGMT-zone VM, with Shuffle deployed via Docker and TheHive/Cortex installed as native system services.
- **Native Active Response** – Wazuh’s Active Response module allows arbitrary scripts (including webhook calls) to be fired directly from a manager-side rule match, which is the trigger mechanism this project relies on.
- **Full incident lifecycle coverage** – unlike Splunk alone, the new stack covers detection (Wazuh), orchestration (Shuffle), documentation (TheHive), and enrichment (Cortex) in a single connected pipeline, closely mirroring how a real SOC operates.

The migration plan, host inventory, and phased cut-over strategy were documented in a structured internal migration document ([SOC_Lab_XDR_SOAR_Migration.docx](#)) prior to implementation.

Table 3: Tool selection rationale

Tool	Why it was selected
Wazuh	Detection engine, endpoint telemetry collection, and the only component of the stack with a native Active Response mechanism capable of triggering an external webhook on rule match.
Shuffle	Free, self-hostable workflow orchestrator with first-class webhook and HTTP-request nodes, avoiding the licensing cost of commercial SOAR platforms.
TheHive	Purpose-built incident/case management with a mature REST API, letting Shuffle create, update, and resolve cases programmatically.
Cortex	Native pairing with TheHive; provides a unified analyzer interface over VirusTotal, AbuseIPDB, and OTX rather than scripting each API individually.
OPNsense	Already the perimeter firewall for the lab; its REST API allows alias-based blocklists to be updated without manual rule editing.
Cowrie	Generates realistic, fully-logged attacker interaction (credentials, commands, session data) that a synthetic test script would not produce.

A Note on “XDR”

The XDR capability described in this report is an **architectural outcome** of integrating multiple telemetry, detection, orchestration, enrichment, and response components – not a claim that any single product (Wazuh included) is a commercial XDR platform on its own. Wazuh provides the detection and endpoint telemetry layer; the “extended” scope comes from correlating that telemetry with orchestration (Shuffle), case management (TheHive), enrichment (Cortex), and network-layer response (OPNsense) into one connected pipeline.

5 Infrastructure Deployment

5.1 Wazuh Manager and Agent Fleet

The Wazuh manager (192.168.107.20) centralizes telemetry from seven agents covering every zone of the lab: two Windows hosts (client and domain controller), a Kali attacker box, a Suricata IDS sensor, the DVWA web server, the WAF, and the Cowrie honeypot.

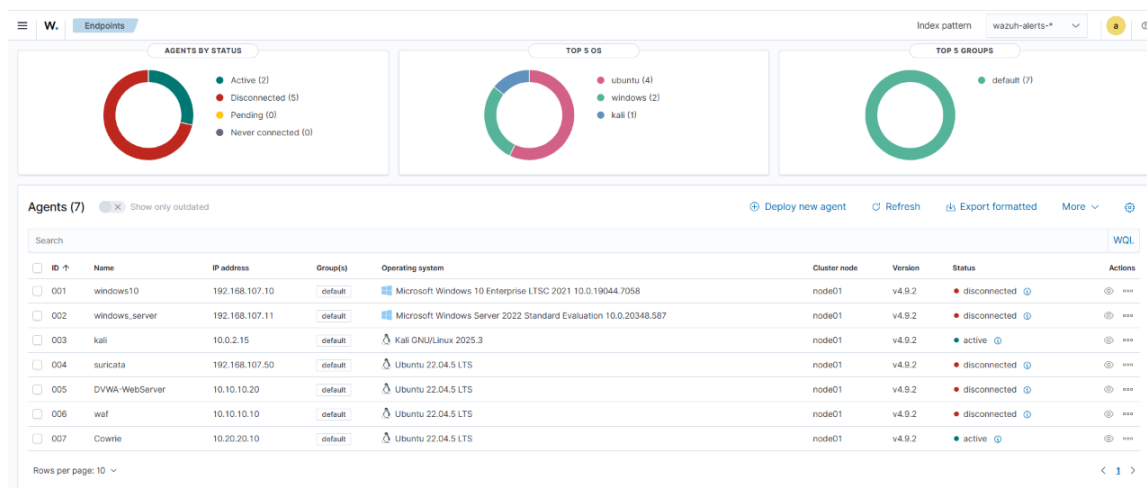


Figure 2: Wazuh Endpoints view: 7 registered agents across Windows, Ubuntu, and Kali operating systems, distributed across the **default** agent group.

```
may@wazuh:~$ sudo /var/ossec/bin/agent_control -l

Wazuh agent_control. List of available agents:
  ID: 000, Name: wazuh (server), IP: 127.0.0.1, Active/Local
  ID: 001, Name: windows10, IP: any, Disconnected
  ID: 002, Name: windows_server, IP: any, Disconnected
  ID: 003, Name: kali, IP: any, Disconnected
  ID: 004, Name: suricata, IP: any, Disconnected
  ID: 005, Name: DVWA-WebServer, IP: any, Disconnected
  ID: 006, Name: waf, IP: any, Disconnected
  ID: 007, Name: Cowrie, IP: any, Active

List of agentless devices:
```

Figure 3: CLI verification of agent status via `agent_control -l` on the Wazuh manager, confirming registration of all seven agents (000–007).

5.2 Log Source Coverage

Each monitored host was validated individually in the Wazuh *Threat Hunting* module to confirm log ingestion before wiring up automated response.

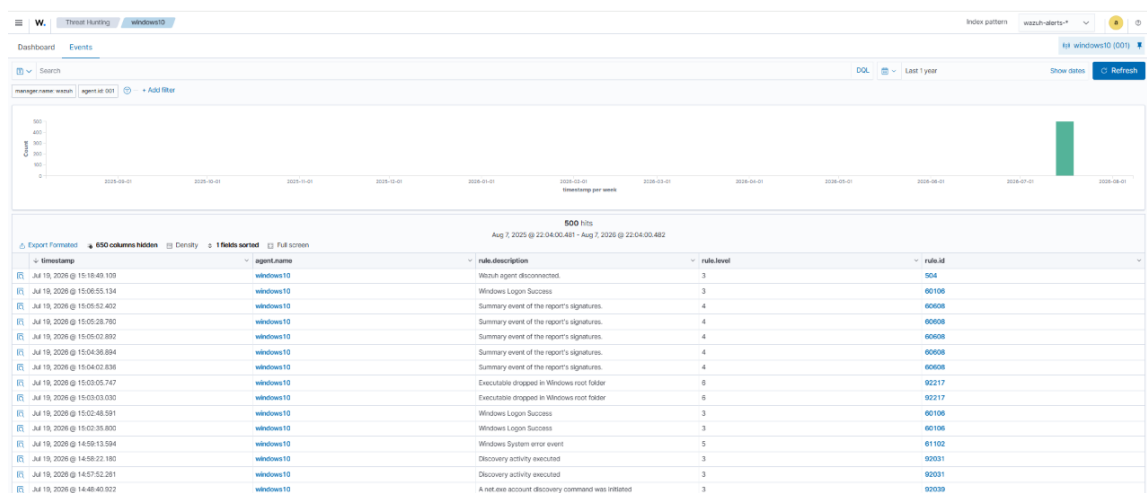


Figure 4: Wazuh events for the Windows 10 client agent (500 hits over the past year), including logon events, discovery activity, and executable-drop detections.

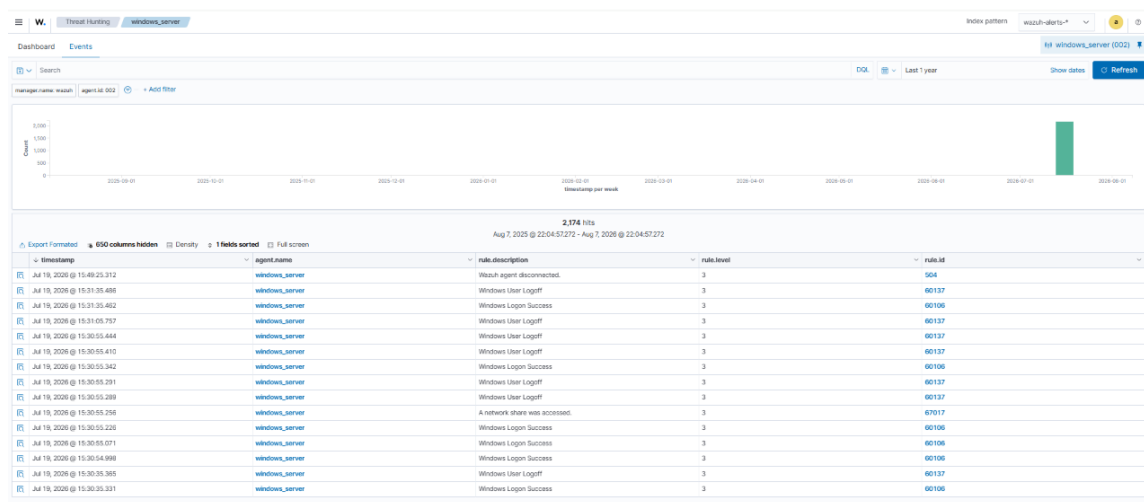


Figure 5: Wazuh events for the Windows Server 2022 domain controller agent (2,174 hits), including logon/logoff and network share access events.

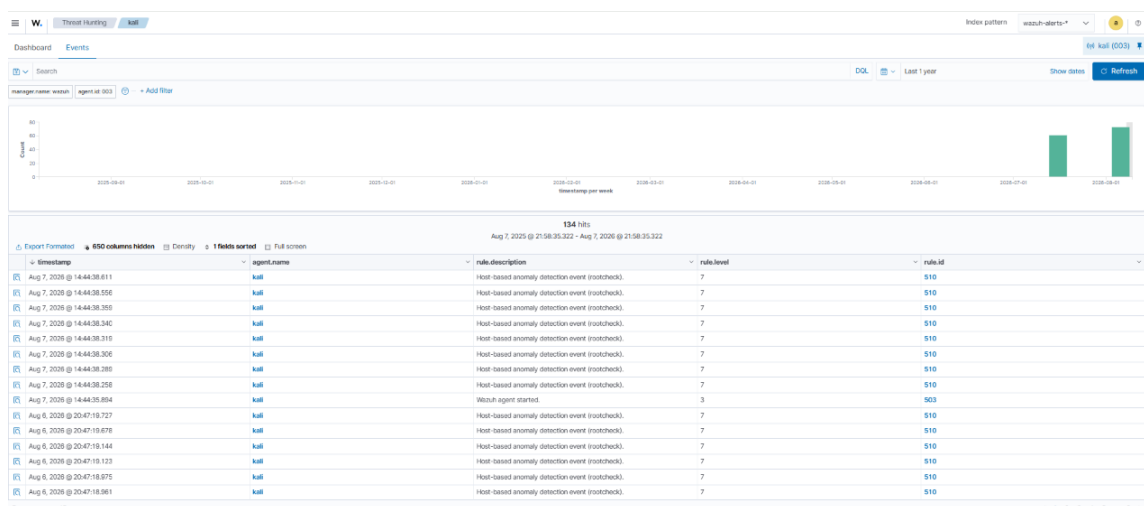


Figure 6: Wazuh events for the Kali attacker agent, primarily host-based anomaly detections (rootcheck).

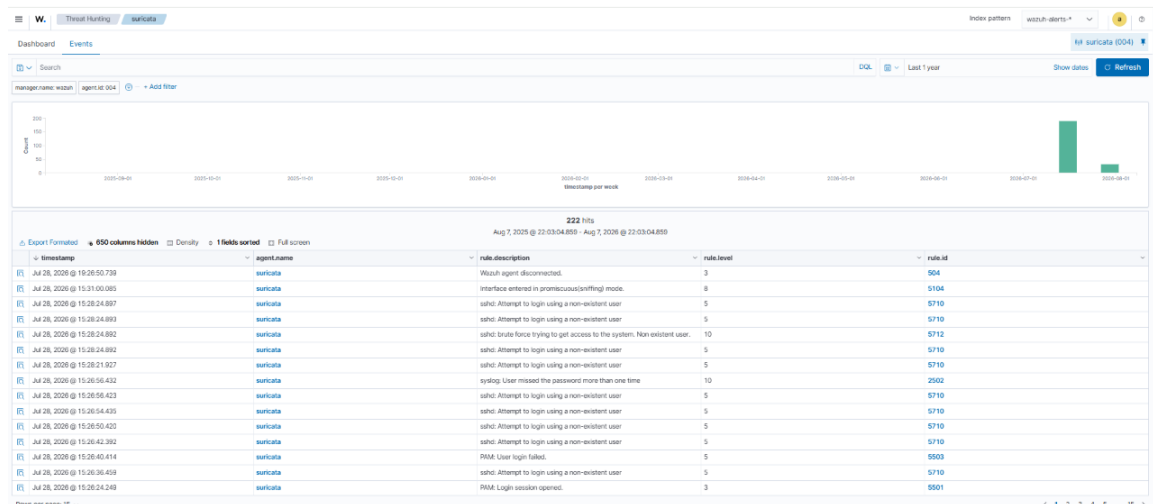


Figure 7: Wazuh events ingested from the Suricata IDS sensor, showing SSH brute-force and non-existent-user login attempts (rule IDs 5710/5712).

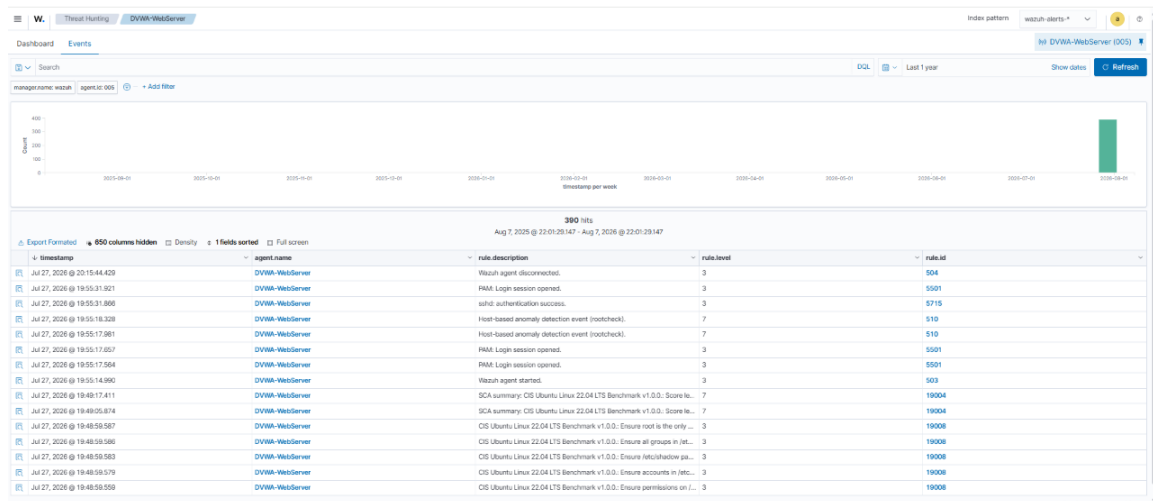


Figure 8: Wazuh events for the DVWA web server agent (390 hits), including SCA (Security Configuration Assessment) findings against the CIS Ubuntu 22.04 benchmark.

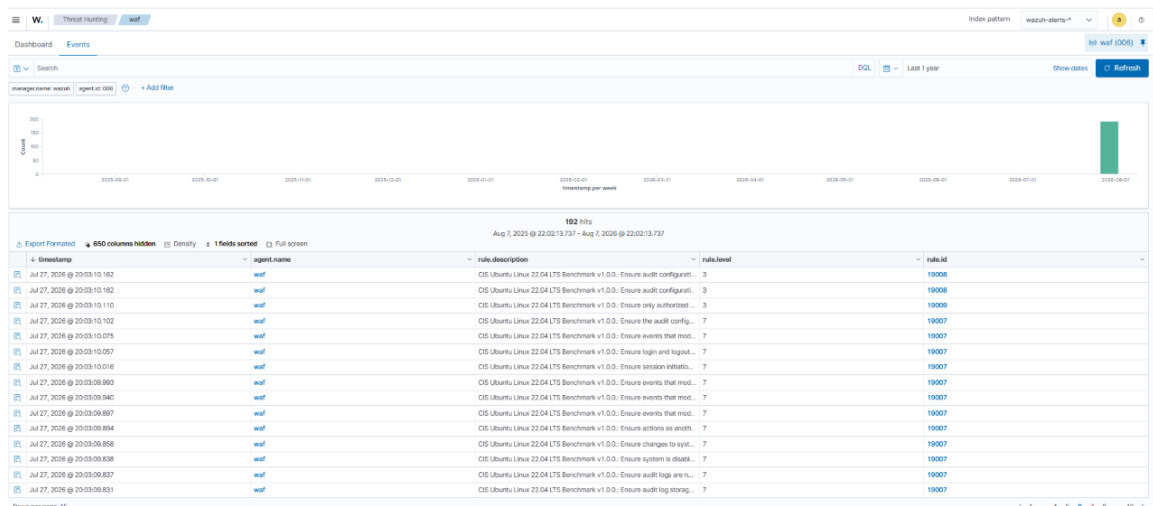


Figure 9: Wazuh events for the WAF agent (192 hits), predominantly CIS Ubuntu benchmark compliance findings.

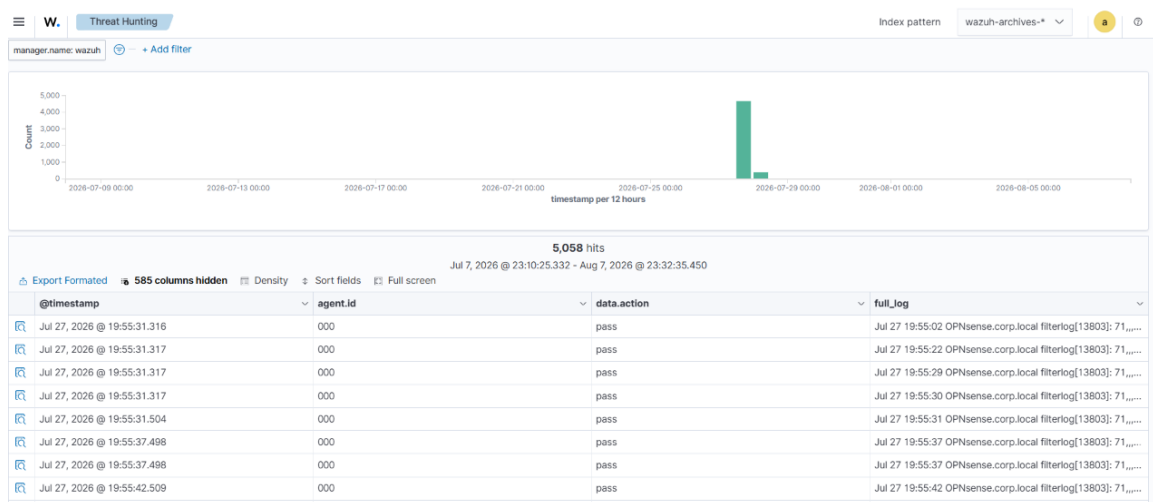


Figure 10: OPNsense firewall pass/deny events forwarded into and visualized within the Wazuh wazuh-archives-* index.

6 Automated Response Pipeline Design

6.1 Pipeline Flow

The end-to-end pipeline was designed around a single trigger: a Wazuh rule match on the Cowrie honeypot (`cowrie.login.success`, rule 100100) or an sshd brute-force detection (rule 5712). Once triggered, Wazuh's Active Response module executes a script that calls a Shuffle webhook, and Shuffle takes over the rest of the workflow autonomously.

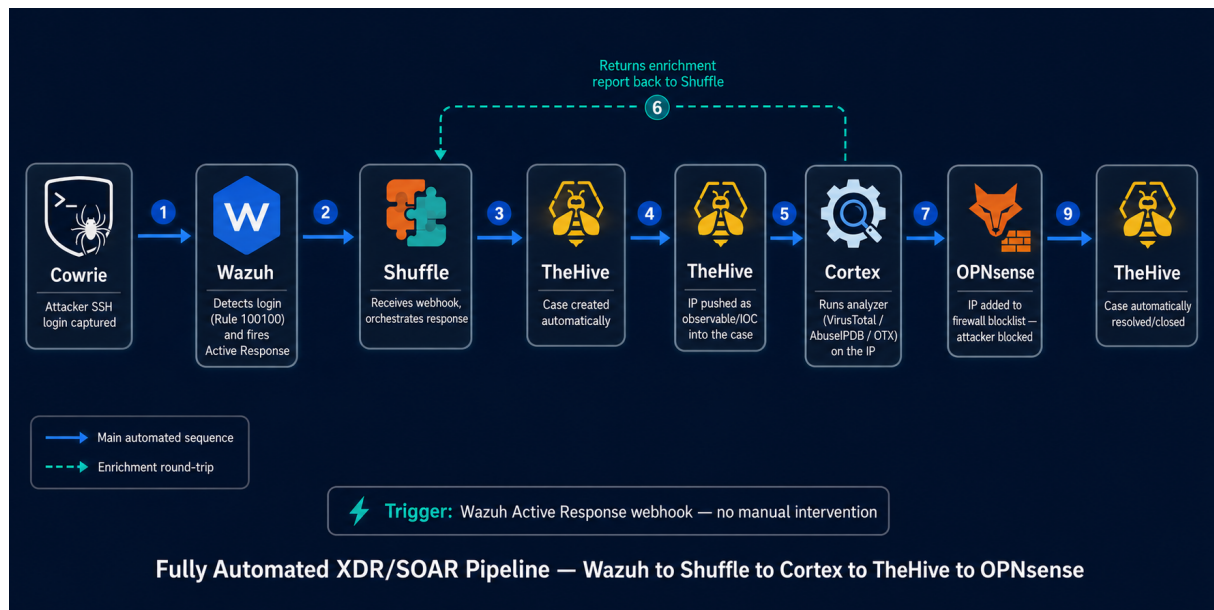


Figure 11: Fully automated XDR/SOAR pipeline: Cowrie captures the login → Wazuh detects and fires Active Response → Shuffle receives the webhook and orchestrates the response → a case is created in TheHive → the offending IP is pushed as an observable → Cortex runs enrichment analyzers → OPNsense blocks the IP → TheHive case is auto-resolved.

6.2 Active Response Configuration

The Active Response block on the Wazuh manager was configured to invoke a custom script, `shuffle-webhook.sh`, on any match of rule 100100 (*Cowrie: Successful honeypot login*) or rule 5712 (*sshd brute-force*). The script's job is simply to relay the raw Wazuh alert JSON to the Shuffle webhook endpoint via `curl`.

Cowrie: Successful honeypot login (credential accepted)					View alerts of this Rule
Information					
ID	Level	File	Path		
100100	5	local_rules.xml	etc/rules		
Groups					
cowrie, authentication_success, cowrie					
Details					
Related rules					
ID	Description	Groups	Compliance	Level	File
100001	sshd: authentication failed from IP 1.1.1.1	authentication_failed, local_logins, sshd	FAIL	5	local_rules.xml
100100	Cowrie: Successful honeypot login (credential accepted)	cowrie, authentication_success, cowrie		5	local_rules.xml
100101	Cowrie: Failed honeypot login attempt	cowrie, authentication_failed, cowrie		3	local_rules.xml
100102	Cowrie: Command entered in honeypot session	cowrie, command_execution, cowrie		8	local_rules.xml
100103	Cowrie: Honeypot session closed	cowrie, session_closed, cowrie		3	local_rules.xml

Figure 12: Wazuh rule detail for rule 100100 (*Cowrie: Successful honeypot login*), showing its related rule family covering failed logins, command execution, and session closure.

For clarity, the relevant `ossec.conf` blocks on the Wazuh manager are reproduced below (command definition and the active-response binding, using the corrected `server` location discussed in Section 6.3):

Listing 1: Wazuh manager: command definition and active-response binding used to trigger the Shuffle webhook

```

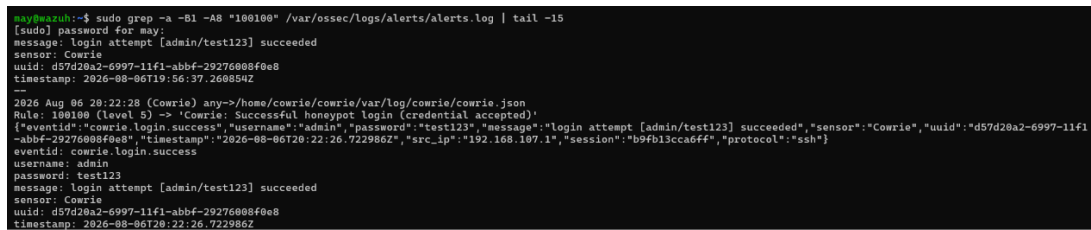
1 <command>
2   <name>shuffle-webhook</name>
3   <executable>shuffle-webhook.sh</executable>
4   <timeout_allowed>no</timeout_allowed>
5 </command>
6
7 <active-response>
8   <disabled>no</disabled>
9   <command>shuffle-webhook</command>
10  <location>server</location>
11  <rules_id>100100,5712</rules_id>
12 </active-response>

```

6.3 Debugging Journey: the <location> Dispatch Bug

The most significant engineering challenge in this project was a silent failure in the Active Response dispatch path. Wazuh's `ossec.conf` active-response block supports a `<location>` directive that determines *where* the response script executes: `local` (on the manager, in response to a locally generated alert) versus `server` (dispatched centrally regardless of which agent generated the alert). Because the Cowrie honeypot alert originates from an agent rather than the manager itself, the Active Response initially never fired when configured with the wrong location semantics – the script was queued but never actually executed, with no explicit error surfaced in the logs.

Diagnosing this required stepping through the full Active Response queue by hand: repeatedly triggering failed SSH logins, tailing `/var/ossec/logs/active-responses.log`, and confirming whether the `shuffle-webhook.sh` script was actually invoked.



```

may@wazuh:~$ sudo grep -a -B1 -A8 "100100" /var/ossec/logs/alerts/alerts.log | tail -15
[sudo] password for may:
message: login attempt [admin/test123] succeeded
sensor: Cowrie
uid: d57d20a2-6997-11f1-abbf-29276080f9e8
timestamp: 2026-08-06T19:56:37.260854Z
--
2026 Aug 06 20:22:28 (Cowrie) any->/home/cowrie/cowrie/var/log/cowrie/cowrie.json
Rule: 100100 (level 5) -> 'Cowrie: Successful honeypot login (credential accepted)'
{"eventid":"cowrie.login.success","username":"admin","password":"test123","message":"login attempt [admin/test123] succeeded","sensor":"Cowrie","uid":"d57d20a2-6997-11f1-abbf-29276080f9e8","timestamp":"2026-08-06T20:22:26.722986Z","src_ip":"192.168.107.1","session":"b9fb13cca6ff","protocol":"ssh"}
eventid: cowrie.login.success
username: admin
password: test123
message: login attempt [admin/test123] succeeded
sensor: Cowrie
uid: d57d20a2-6997-11f1-abbf-29276080f9e8
timestamp: 2026-08-06T20:22:26.722986Z

```

Figure 13: Manual reproduction of a brute-force condition on the Wazuh host: five failed SSH attempts against a non-existent user, followed by inspection of the active-response log after truncation, confirming the Active Response script fired correctly and dispatched the alert payload.

```

may@wazuh:~$ sudo truncate -s 0 /var/ossec/logs/active-responses.log
sshpass -p "test123" ssh admin@10.20.20.10 -p 2222 -o StrictHostKeyChecking=no -o ConnectTimeout=3 exit
sleep 10
sudo cat /var/ossec/logs/active-responses.log
=====
Thu Aug 6 08:31:43 PM UTC 2026
Started script
RAW INPUT:
{"version":1,"origin":{"name":"node01","module":"wazuh-execd"},"command":"add","parameters":{"extra_args":[],"alert":{"timestamp":"2026-08-06T20:31:43.583+0800","rule":{"level":5,"description":"Cowrie: Successful honeypot login (credential accepted)","id":"100100","firedtimes":2,"mail":false,"groups":["cowrie"],"cowrie":{"authentication_success":{"agent":{"id":"007","name":"Cowrie","ip":"10.20.20.10"},"manager":{"name":"wazuh","id":"1786948393.486759"},"full_log":{"eventid":"Cowrie login success"},"username":"admin","password":"test123"},"message":"login attempt [admin/test123] succeeded"},"sensor":{"cowrie":{"uid":"d57d28a2-6997-11f1-abbf-29276008f9e8"},"timestamp":"2026-08-06T20:31:42.678850Z"},"src_ip":"192.168.107.20"},"session":{"20666dd088a9"},"protocol":"ssh"},"decoder":{"name":"json"},"data":{"protocol":"ssh","eventid":"Cowrie login success","username":"admin","password":"test123","message":"login attempt [admin/test123] succeeded"},"sensor":{"cowrie":{"uid":"d57d28a2-6997-11f1-abbf-29276008f9e8"},"timestamp":"2026-08-06T20:31:42.678850Z"},"src_ip":"192.168.107.20"},"session":"20666dd088a9"},"location":"/home/cowrie/cowrie/var/log/cowrie/cowrie.json"},"program":"active-response/bin/shuffle-webhook.sh"}}
SRCIP=192.168.107.20
Note: Unnecessary use of -X or --request, POST is already inferred.
* Trying 10.30.30.30:3001...
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
 0   0    0     0    0     0      0  0.00:00  0.00:00  0.00:00  0.00:00  0.00:00  0
 > POST /api/v1/hooks/webhook_98e16012-343c-4a44-bd33-ea9adedb6656 HTTP/1.1
 > Host: 10.30.30.30:3001
 > User-Agent: curl/7.81.0
 > Accept: */*
 > Content-Type: application/json
 > Content-Length: 23
 >
 } [23 bytes data]
 100  23    0     0    0     0      0  0.00:01  0.00:01  0.00:01  0.00:01  0.00:01  18* Mark bundle as not supporting multiuse
 > HTTP/1.1 200 OK
 < Server: nginx/1.29.3
 < Date: Thu, 06 Aug 2026 20:31:47 GMT
 < Content-Type: application/json
 < Content-Length: 73
 < Connection: keep-alive
 < Vary: Accept-Encoding
 <
 } [73 bytes data]
 100  96 100    73 100    23    21    6 0.00:03  0.00:03  0.00:03  0.00:03  0.00:03  28
 * Connection #0 to host 10.30.30.30 left intact
 {"success": true, "execution_id": "6608888f-2e93-41b3-91b3-73d358867f9c"}Finished curl
may@wazuh:~$

```

Figure 14: Successful Active Response execution against the Cowrie honeypot: the script correctly extracts the source IP (192.168.107.20) and posts the alert JSON to the Shuffle webhook, receiving an HTTP 200 with {"success": true}.

Root Cause Summary

The `<location>server</location>` directive is required for agent-originated alerts (such as the Cowrie honeypot, which reports through an agent) so that the manager dispatches the response centrally rather than expecting the response to run locally on the agent – which, for a honeypot appliance, is neither possible nor desirable. Once corrected, every matching alert reliably triggered the webhook end-to-end.

6.4 Attacker Interaction Capture

To validate that the honeypot itself was generating realistic, parseable telemetry, raw Cowrie JSON logs and SSH client interaction were inspected directly.

```

(kali@kali)~$ sshpass -p "test123" ssh admin@10.20.20.10 -p 2222 -o StrictHostKeyChecking=no -o ConnectTimeout=3 exit
(kali@kali)~$

```

Figure 15: Simulated attacker SSH login from the Kali host into the Cowrie honeypot (10.20.20.10:2222) using `sshpass`.

```

cowrie:~$ sudo tail -10 /home/cowrie/cowrie/var/log/cowrie/cowrie.json
[sudo] password for may:
{"eventid": "cowrie.session.connect", "src_ip": "192.168.107.1", "src_port": 54568, "dst_ip": "10.20.20.10", "dst_port": 2222, "session": "b9fb13cca6ff", "protocol": "ssh", "message": "New connection: 192.168.107.1:54568 (10.20.20.10:2222) [session: b9fb13cca6ff]", "sensor": "Cowrie", "uuid": "d57d20a2-6997-11f1-abbf-29276080f0e8", "timestamp": "2026-08-06T20:22:26.590914Z"}
{"eventid": "cowrie.client.version", "version": "SSH-2.0-OpenSSH.10.0p2 Debian-8", "message": "Remote SSH version: SSH-2.0-OpenSSH.10.0p2 Debian-8", "sensor": "Cowrie", "uuid": "d57d20a2-6997-11f1-abbf-29276080f0e8", "timestamp": "2026-08-06T20:22:26.607912Z", "src_ip": "192.168.107.1", "session": "b9fb13cca6ff", "protocol": "ssh"}
{"eventid": "cowrie.client.kex", "hash": "eeca2460550b9d0d084ecf2f70a75356", "hash_algorithm": "mlkem768x25519-sha256", "sntrup761x25519-sha512", "sntrup761x25519-sha512@openssh.com", "curve25519-sha256", "curve25519-sha256@libssh.org", "ecdh-sha2-nistp256", "ecdh-sha2-nistp384", "ecdh-sha2-nistp521", "diffie-hellman-group-exchange-sha256", "diffie-hellman-group16-sha512", "diffie-hellman-group18-sha512", "diffie-hellman-group14-sha256", "ext-info-c", "kex-strict-c-v00@openssh.com", "chacha20-poly1305@openssh.com", "aes128-gcm@openssh.com", "aes256-gcm@openssh.com", "aes128-ctr", "aes192-ctr", "aes256-ctr", "umac-64-etm@openssh.com", "umac-128-etm@openssh.com", "umac-64@openssh.com", "umac-128@openssh.com", "hmac-sha2-256", "hmac-sha2-512", "hmac-sha1", "hmac-sha1-etm@openssh.com", "sk-ecdsa-sha2-nistp256-cert-v01@openssh.com", "sk-ecdsa-sha2-nistp384-cert-v01@openssh.com", "sk-ecdsa-sha2-nistp521-cert-v01@openssh.com", "sk-ecdsa-sha2-nistp256@openssh.com", "rsa-sha2-512", "rsa-sha2-256", "rsa-sha2-512", "rsa-sha2-256-cert-v01@openssh.com", "ssh-ed25519", "ecdsa-sha2-nistp256", "ecdsa-sha2-nistp384", "ecdsa-sha2-nistp521", "sk-ssh-ed25519@openssh.com", "sk-ecdsa-sha2-nistp256@openssh.com", "rsa-sha2-512", "rsa-sha2-256", "encCS": [{"c": "hacha20-poly1305@openssh.com", "aes128-gcm@openssh.com", "aes256-gcm@openssh.com", "aes128-ctr", "aes192-ctr", "aes256-ctr"}, {"macCS": [{"umac-64-etm@openssh.com", "umac-128-etm@openssh.com", "hmac-sha2-256-etm@openssh.com", "hmac-sha2-512-etm@openssh.com", "hmac-sha1-etm@openssh.com", "umac-64@openssh.com", "umac-128@openssh.com", "hmac-sha2-256", "hmac-sha2-512", "hmac-sha1"}, {"compCS": [{"none", "zlib@openssh.com"}], "langCS": [{""]}]}], "message": "SSH client hash fingerprint: eeca2460550b9d0d084ecf2f70a75356", "sensor": "Cowrie", "uuid": "d57d20a2-6997-11f1-abbf-29276080f0e8", "timestamp": "2026-08-06T20:22:26.626413Z", "src_ip": "192.168.107.1", "session": "b9fb13cca6ff", "protocol": "ssh"}
{"eventid": "cowrie.login.success", "username": "admin", "password": "test123", "message": "login attempt [admin/test123] succeeded", "sensor": "Cowrie", "uuid": "d57d20a2-6997-11f1-abbf-29276080f0e8", "timestamp": "2026-08-06T20:22:26.722986Z", "src_ip": "192.168.107.1", "session": "b9fb13cca6ff", "protocol": "ssh"}
{"eventid": "cowrie.client.var", "name": "COLORTERM", "value": "truecolor", "message": "request env: COLORTERM=truecolor", "sensor": "Cowrie", "uuid": "d57d20a2-6997-11f1-abbf-29276080f0e8", "timestamp": "2026-08-06T20:22:26.755252Z", "src_ip": "192.168.107.1", "session": "b9fb13cca6ff", "protocol": "ssh"}
{"eventid": "cowrie.client.var", "name": "LANG", "value": "en_US.UTF-8", "message": "request env: LANG=en_US.UTF-8", "sensor": "Cowrie", "uuid": "d57d20a2-6997-11f1-abbf-29276080f0e8", "timestamp": "2026-08-06T20:22:26.758673Z", "src_ip": "192.168.107.1", "session": "b9fb13cca6ff", "protocol": "ssh"}
{"eventid": "cowrie.session.params", "arch": "linux-x64-lsb", "message": [], "sensor": "Cowrie", "uuid": "d57d20a2-6997-11f1-abbf-29276080f0e8", "timestamp": "2026-08-06T20:22:28.995886Z", "src_ip": "192.168.107.1", "session": "b9fb13cca6ff", "protocol": "ssh"}
{"eventid": "cowrie.command.input", "input": "exit", "message": "CMD: exit", "sensor": "Cowrie", "uuid": "d57d20a2-6997-11f1-abbf-29276080f0e8", "timestamp": "2026-08-06T20:22:28.999156Z", "src_ip": "192.168.107.1", "session": "b9fb13cca6ff", "protocol": "ssh"}
{"eventid": "cowrie.log.closed", "ttylog": "var/lib/cowrie/tty/e596899f114b5162402325dfb31fdaa792fabed718628336cc7a35a24f38eaa9", "size": 0, "shasum": "e596899f114b5162402325dfb31fdaa792fabed718628336cc7a35a24f38eaa9", "duplicate": true, "duration": "0.0", "message": "Closing TTY Log: var/lib/cowrie/tty/e596899f114b5162402325dfb31fdaa792fabed718628336cc7a35a24f38eaa9 after 0.0 seconds", "sensor": "Cowrie", "uuid": "d57d20a2-6997-11f1-abbf-29276080f0e8", "timestamp": "2026-08-06T20:22:28.940159Z", "src_ip": "192.168.107.1", "session": "b9fb13cca6ff", "protocol": "ssh"}
{"eventid": "cowrie.session.closed", "duration": "2.3", "message": "Connection lost after 2.3 seconds", "sensor": "Cowrie", "uuid": "d57d20a2-6997-11f1-abbf-29276080f0e8", "timestamp": "2026-08-06T20:22:28.952454Z", "src_ip": "192.168.107.1", "session": "b9fb13cca6ff", "protocol": "ssh"}

```

Figure 16: Raw Cowrie JSON session log showing the full SSH handshake, key exchange, successful credential-based login (admin/test123), and clean session closure – the exact event that rule 100100 correlates on.

7 Shuffle SOAR Workflow

The Shuffle workflow is triggered by a single webhook node and fans out into six sequential HTTP action nodes, each corresponding to one stage of the incident response lifecycle: case creation, analyzer discovery, analyzer execution, enrichment report retrieval, observable push, IP blocking, and finally case resolution.

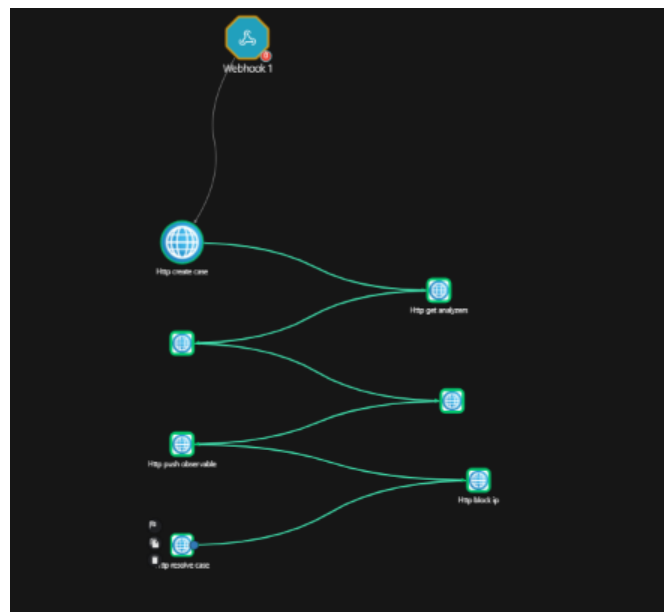


Figure 17: Shuffle workflow graph: a single Webhook trigger fans out through six chained HTTP nodes (*Http create case*, *Http get analyzers*, *Http push observable*, *Http run analyzer*, *Http block ip*, *Http resolve case*).

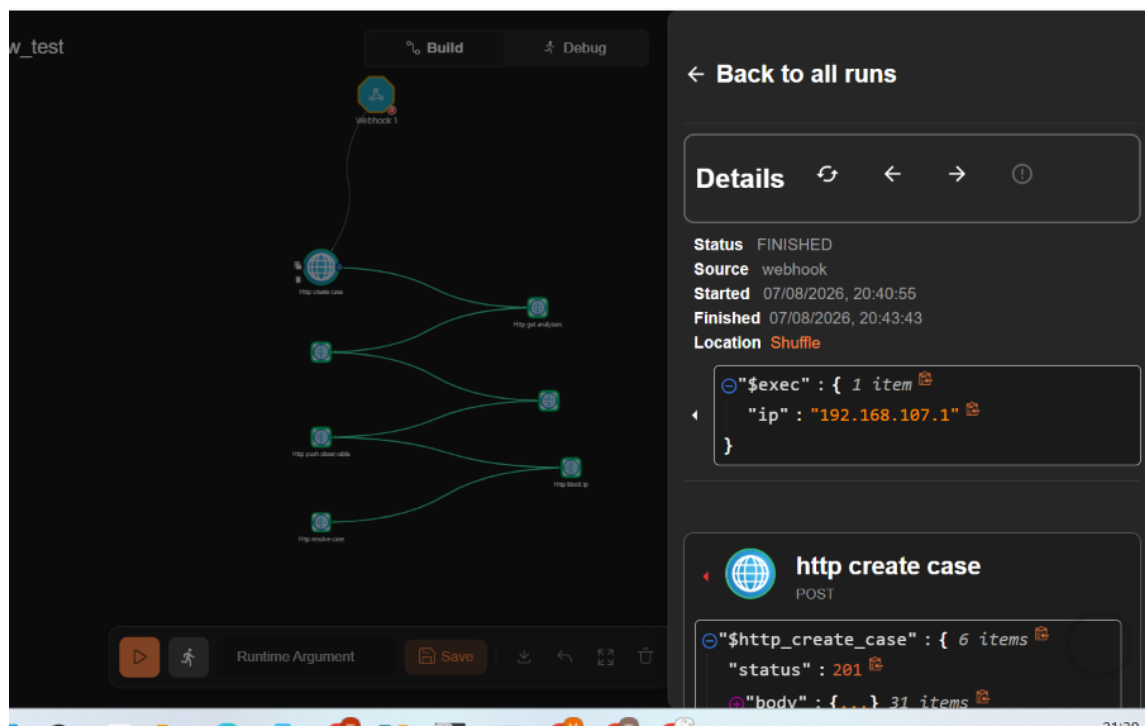


Figure 18: Execution detail view of a completed workflow run: status **FINISHED**, triggered by the **webhook** source, with the extracted attacker IP (192.168.107.1) passed as the **\$exec** variable into the *Http create case* node.

7.1 Workflow Node Breakdown

1. Create Case. The workflow opens with a POST to TheHive's case API, auto-generating a case titled with the offending IP and tagged **shuffle-auto** / **xdr**.

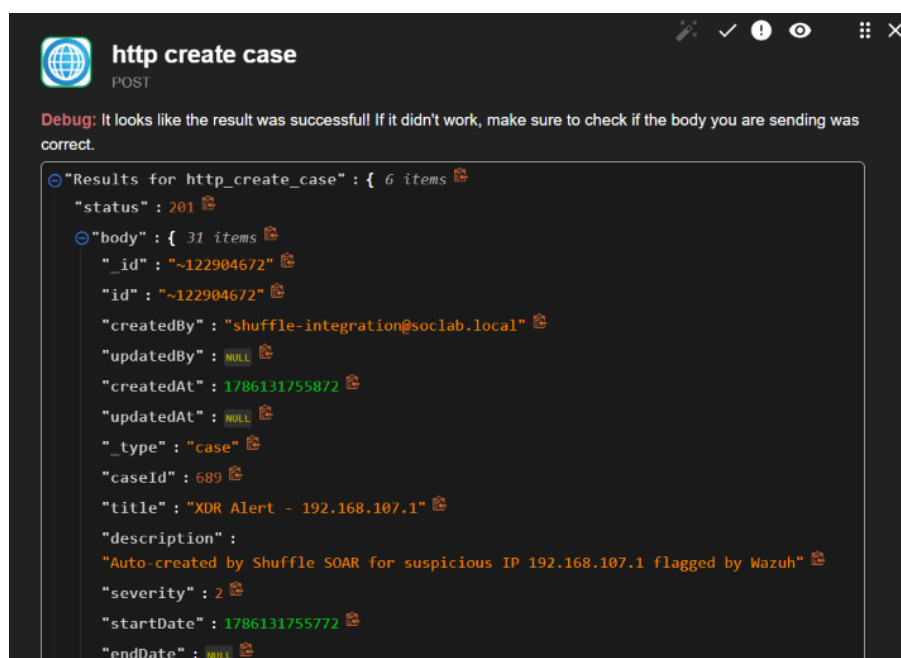


Figure 19: *Http create case* node: HTTP 201 response confirming case #689 was created in TheHive with title “XDR Alert - 192.168.107.1”.

2. Discover Available Analyzers. Before running enrichment, the workflow queries Cortex for the list of enabled analyzers so the correct worker IDs can be referenced dynamically.

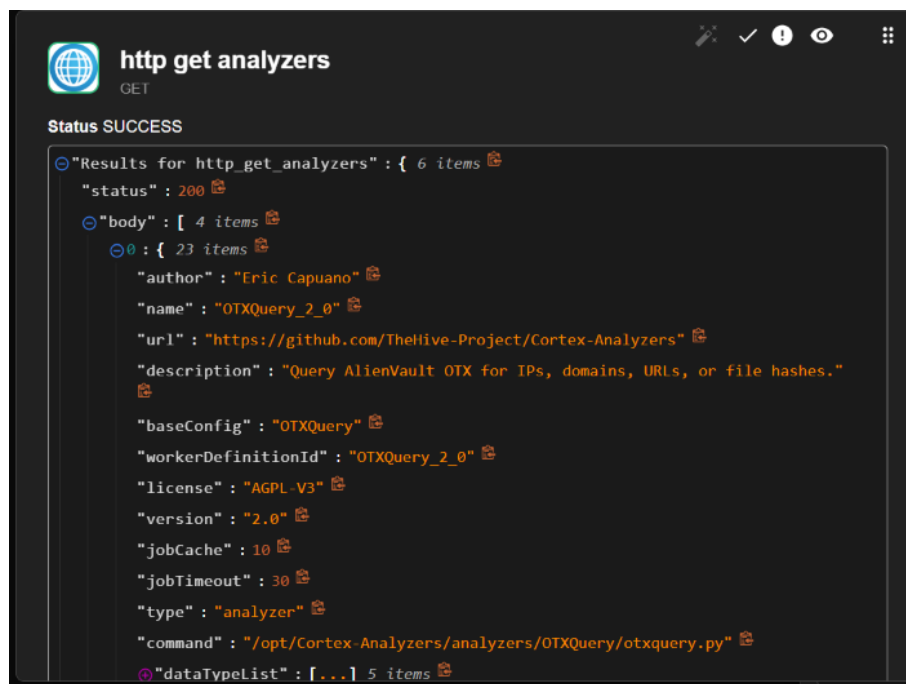


Figure 20: *Http get analyzers* node: Cortex returns the configured analyzer catalog, including OTXQuery_2_0 for AlienVault OTX lookups.

3. Run Analyzer. The IP observable is submitted to the selected Cortex analyzer for asynchronous processing.

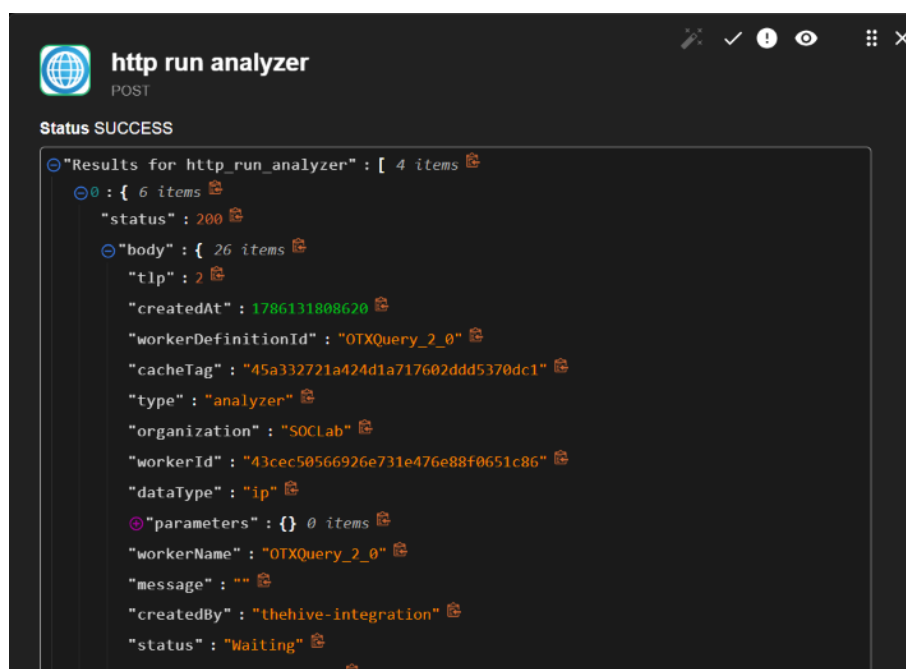


Figure 21: *Http run analyzer* node: analyzer job queued against Cortex with status **Waiting** for the target IP 192.168.107.1.

4. Retrieve Enrichment Report. Once the analyzer job completes, the workflow polls

for and retrieves the enrichment report.

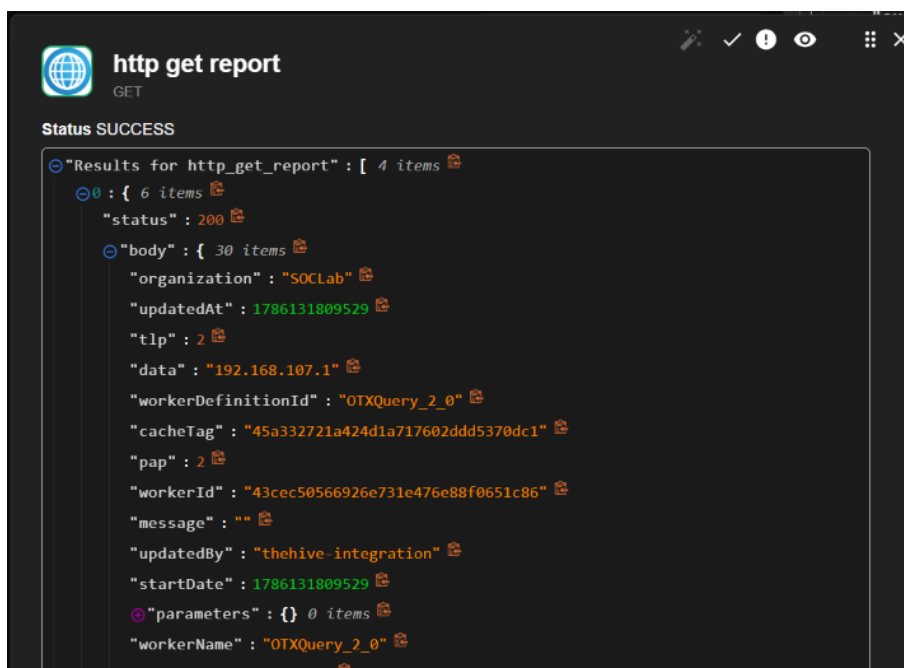


Figure 22: *Http get report* node: Cortex job report returned with status *InProgress*/complete, including worker metadata for the OTXQuery analyzer.

5. Push Observable. The enriched IP is attached to the TheHive case as a case artifact/observable.

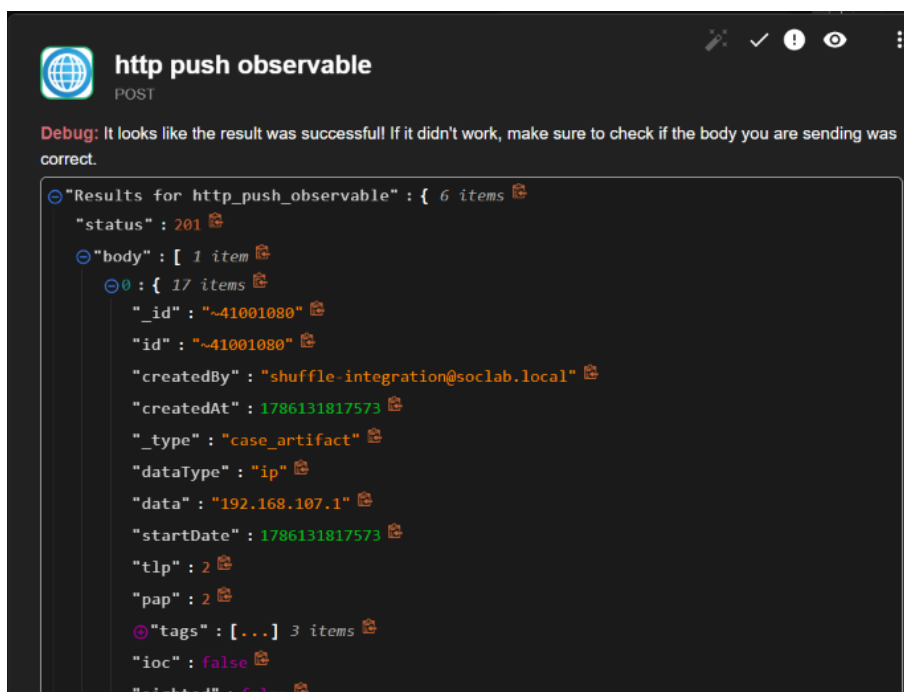


Figure 23: *Http push observable* node: HTTP 201 confirming the IP 192.168.107.1 was added as a case artifact in TheHive.

6. Block IP. The workflow calls the OPNsense firewall API to append the attacker IP to a dedicated blocklist alias.

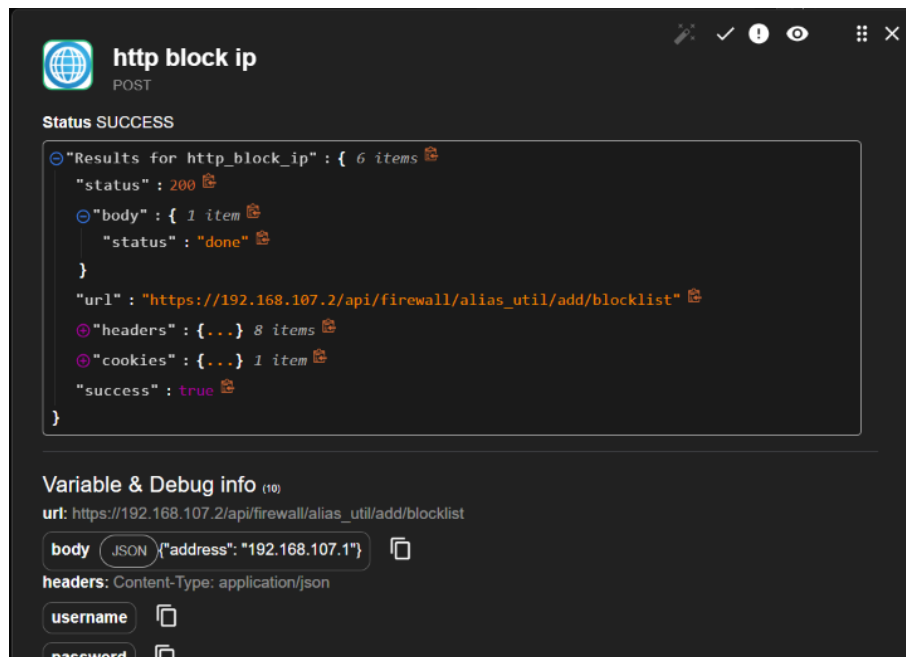


Figure 24: *Http block ip* node: successful POST to the OPNsense `/api/firewall/alias_util/add/blocklist` endpoint, adding 192.168.107.1 to the blocklist.

7. Resolve Case. Finally, the case is automatically marked resolved in TheHive, closing the loop without analyst intervention.

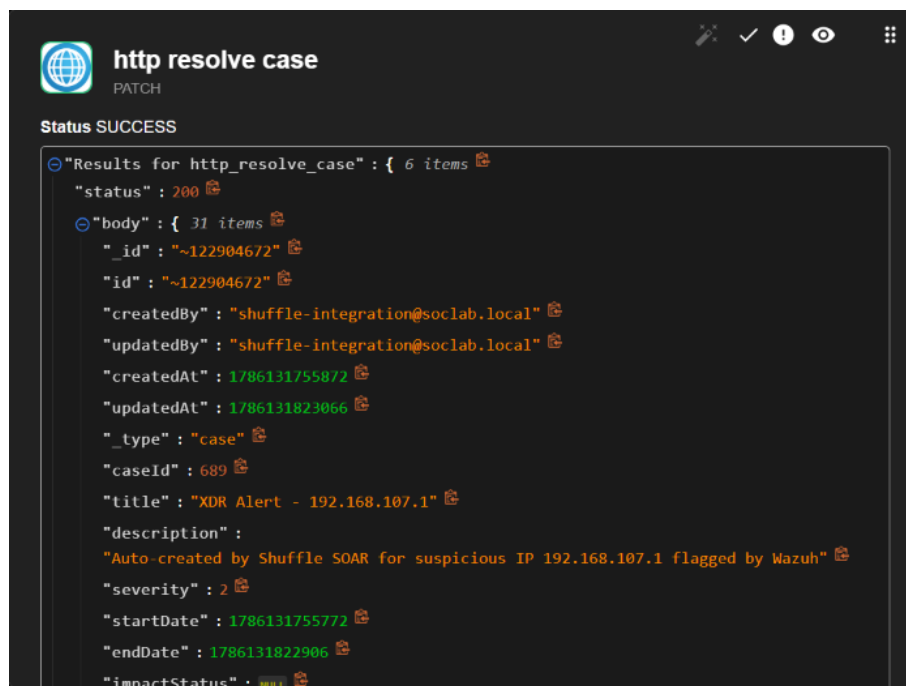


Figure 25: *Http resolve case* node: PATCH request confirming case #689 was updated and closed by the Shuffle integration account.

8 TheHive Case Management

TheHive was configured as the central case repository, with a dedicated service account (`shuffle-integration@soclab.local`) used by Shuffle to create, update, and resolve cases programmatically, alongside

a human analyst account and a Wazuh integration service account.

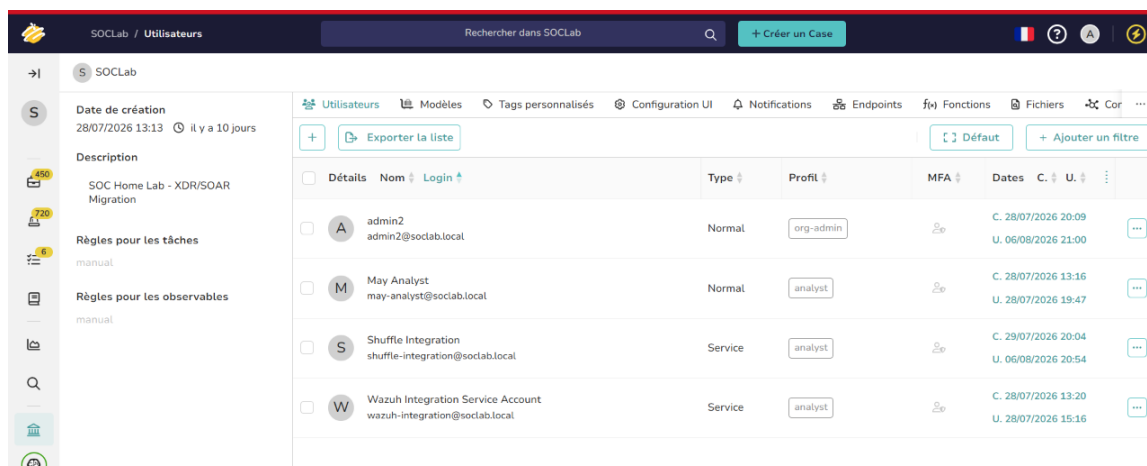


Figure 26: TheHive Users administration view: separate accounts for the human analyst (*May Analyst*), and dedicated service accounts for the Shuffle and Wazuh integrations, both scoped to the **analyst** profile.

A reusable case template (**XDR-Alert**) standardizes severity, TLP/PAP classification, and tagging for every automatically generated case.

Édition du modèle de case

Préfixe de titre du modèle de case...

Nom *
XDR-Alert

Nom d'affichage
[XDR] {{title}}

TLP

PAP

Séverité

Tags

Description *

Prévisualiser

Annuler Éditer le modèle de case

Figure 27: TheHive case template “XDR-Alert”: TLP:AMBER / PAP:AMBER, MEDIUM severity default, tagged **wazuh** and **xdr**.

Outbound notifications were configured so that any case event triggers a webhook back to Shuffle, allowing bidirectional automation between the two platforms.

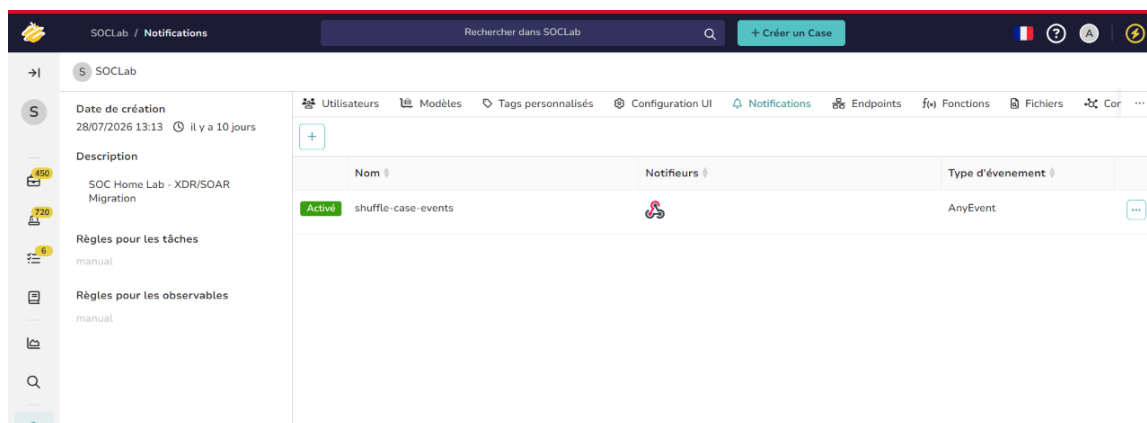


Figure 28: TheHive Notifications configuration: an active `shuffle-case-events` notifier firing on `AnyEvent`, forwarding case activity back into Shuffle.

Analyst tasks are automatically generated on every case (Triage, Enrichment Review, Containment Decision), giving the workflow a documented, auditable trail even though the actions themselves are automated.

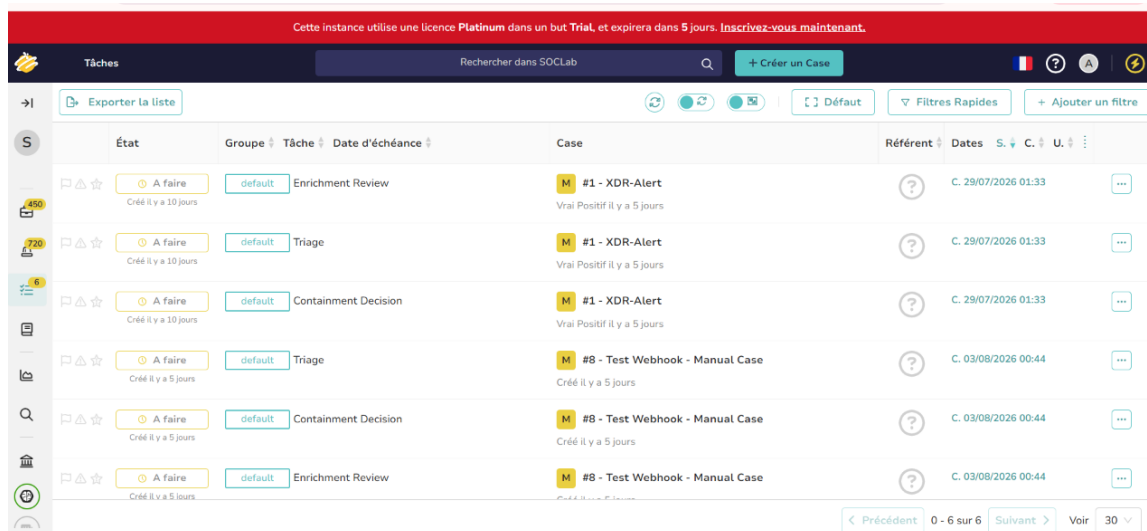
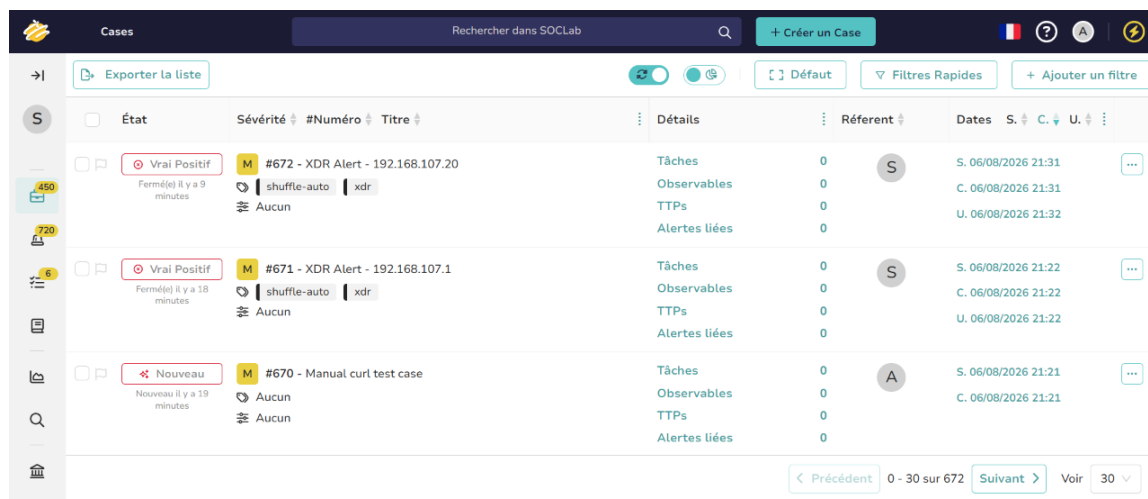


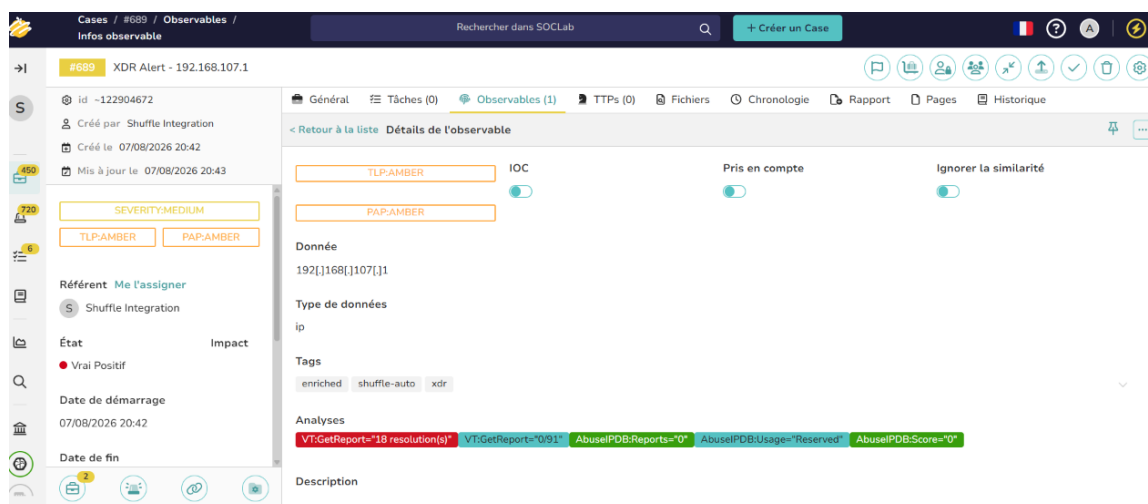
Figure 29: TheHive Tasks queue showing the standard triage/enrichment/containment task set auto-created for both a live brute-force case (#1 - XDR-Alert) and a manual webhook test case (#8).



The screenshot shows a web interface for 'Cases' with a search bar and a '+ Créer un Case' button. Below the header, there's a table of cases. The first two cases are 'Vrai Positif' (True Positive) and the third is 'Nouveau' (New). Each case has a status, severity, number, title, and details. The 'Vrai Positif' cases are marked as 'Fermé(e) il y a 9 minutes' and 'Fermé(e) il y a 18 minutes'. The 'Nouveau' case is marked as 'Nouveau il y a 19 minutes'. The details for each case include 'Tâches' (Tasks), 'Observables' (Observables), 'TTPs' (Tactics, Techniques, and Procedures), and 'Alertes liées' (Linked Alerts). The 'Vrai Positif' cases have 0 tasks, 0 observables, 0 TTPs, and 0 linked alerts. The 'Nouveau' case has 0 tasks, 0 observables, 0 TTPs, and 0 linked alerts. The interface also includes a sidebar with navigation icons and a bottom bar with pagination controls.

État	Sévérité	#Numéro	Titre	Détails	Référent	Dates
Vrai Positif	M	#672 - XDR Alert - 192.168.107.20		Tâches Observables TTPs Alertes liées	0 0 0 0	S. 06/08/2026 21:31 C. 06/08/2026 21:31 U. 06/08/2026 21:32
Vrai Positif	M	#671 - XDR Alert - 192.168.107.1		Tâches Observables TTPs Alertes liées	0 0 0 0	S. 06/08/2026 21:22 C. 06/08/2026 21:22 U. 06/08/2026 21:22
Nouveau	M	#670 - Manual curl test case		Tâches Observables TTPs Alertes liées	0 0 0 0	S. 06/08/2026 21:21 C. 06/08/2026 21:21

Figure 30: TheHive case list showing multiple auto-generated “Vrai Positif” (true positive) cases created and resolved end-to-end by the Shuffle integration in minutes.



The screenshot shows the detail view of a case observable. The left sidebar contains metadata such as 'id', 'Créé par', 'Créé le', 'Mis à jour le', 'Référént', 'État', 'Date de démarrage', and 'Date de fin'. The main content area shows the 'Observables (1)' tab with a table of observables. The first observable is '192[1168][107][1]' with a type of 'ip'. It has tags 'enriched', 'shuffle-auto', and 'xdr'. The 'Analyses' section shows results from VirusTotal and AbuseIPDB. The 'Description' field is empty. The interface also includes a sidebar with navigation icons and a bottom bar with pagination controls.

Observables (1)
TLP:AMBER IOC PAP:AMBER Donnée: 192[1168][107][1] Type de données: ip Tags: enriched, shuffle-auto, xdr Analyses: VT:GetReport="18 resolution(s)" VT:GetReport="0/91" AbuseIPDB:Reports="0" AbuseIPDB:Usage="Reserved" AbuseIPDB:Score="0"

Figure 31: Detail view of an automatically enriched case observable: severity MEDIUM, TLP/-PAP:AMBER, tagged enriched, shuffle-auto, xdr, with VirusTotal and AbuseIPDB analyzer results attached.

9 Cortex Threat Intelligence Enrichment

Cortex hosts the analyzer catalog used for automated IOC enrichment. Three analyzers were configured and enabled for this pipeline: **AbuseIPDB**, **VirusTotal**, and **AlienVault OTX (OTXQuery)**, all scoped to the SOCLab organization with TLP/PAP:AMBER limits and a 10-minute result cache.

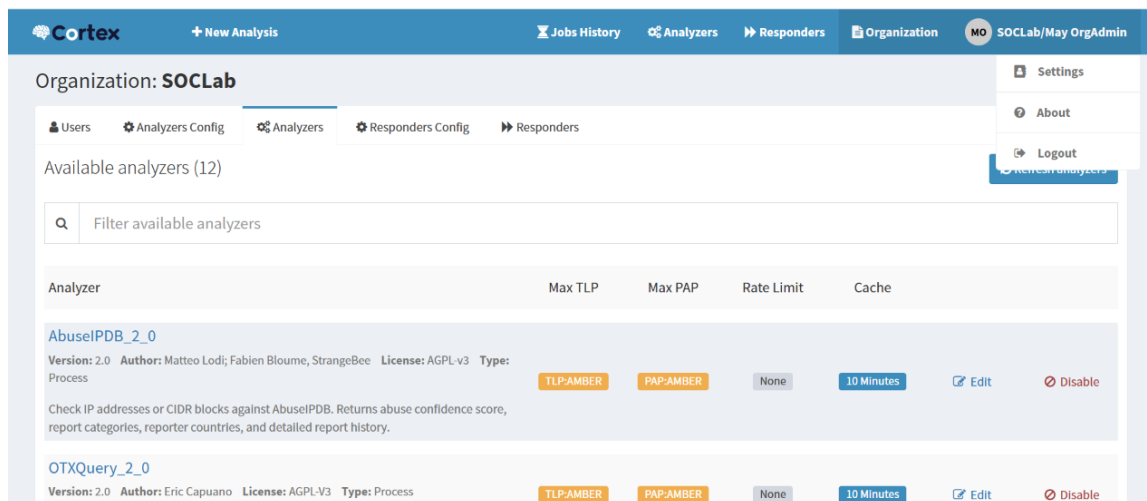


Figure 32: Cortex Analyzers configuration page: 12 available analyzers for the SOCLab organization, with AbuseIPDB_2_0 and OTXQuery_2_0 enabled.

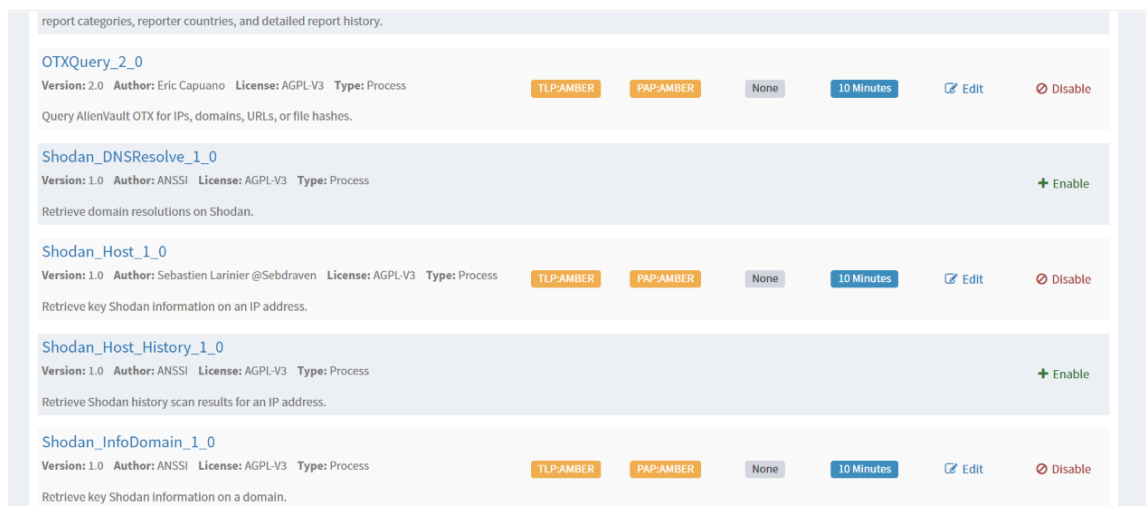


Figure 33: Additional analyzer catalog entries, including Shodan-based analyzers available but not enabled for this pipeline.

Enrichment results are written back into TheHive as observable analyses, giving the analyst (or, in this fully automated case, the audit trail) an immediate confidence score for the attacker IP.

10 OPNsense Automated Containment

The final containment step relies on OPNsense's REST API. A dedicated host alias, **blocklist**, is maintained via the firewall's alias configuration, and the Shuffle workflow appends newly confirmed attacker IPs to it directly through the API – no manual firewall rule editing is required.

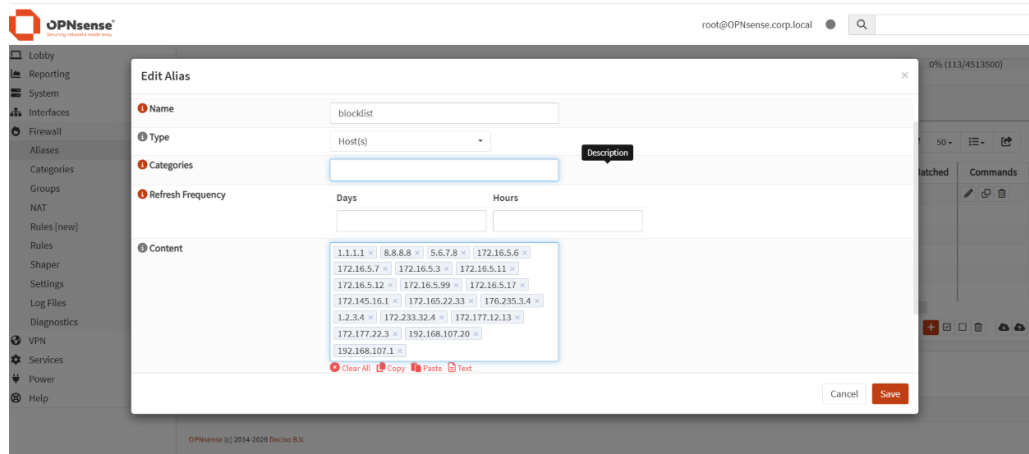


Figure 34: OPNsense *Edit Alias* view for the **blocklist** host alias, showing accumulated attacker IPs appended automatically over multiple pipeline runs, including 192.168.107.1 pushed by the workflow documented above.

11 End-to-End Validation and Test Scenarios

11.1 Proof of End-to-End Execution

The table below consolidates the evidence presented across Sections 5–10 into a single chain-of-custody view, mapping each stage of the pipeline to the specific artifact that proves it executed.

Table 4: End-to-end execution evidence, stage by stage

Stage	Evidence Source	Observed Result
Attack	Cowrie / Kali (sshpas)	Login captured in raw Cowrie JSON
Detection	Wazuh	Rule 100100 / 5712 matched
Trigger	Active Response log	curl POST, HTTP 200, {"success": true}
Orchestration	Shuffle	Workflow run status FINISHED
Case Creation	TheHive	Case created (HTTP 201)
Enrichment	Cortex	VirusTotal / AbuseIPDB reports attached
Containment	OPNsense	IP appended to blocklist alias (HTTP 200)
Closure	TheHive	Case PATCHed to resolved (HTTP 200)

A Note on Source IP Variability

Screenshots throughout this report show several different source addresses for the “attacker” (e.g. 192.168.107.1, 192.168.107.20, and ::1). This is expected: the pipeline was validated across **multiple independent test runs**, and the source address simply reflects where each run originated from – a direct SSH client on the Kali host, a manual reproduction executed locally on the Wazuh host itself (which registers as the IPv6 loopback ::1), and cross-host network tests against other lab VMs. Every run is otherwise self-consistent: each figure’s case ID, IP, and enrichment data correspond to that single run, and no two screenshots in the same figure group mix data from different runs. The variability reflects the number of validation passes performed, not an inconsistency in the pipeline itself.

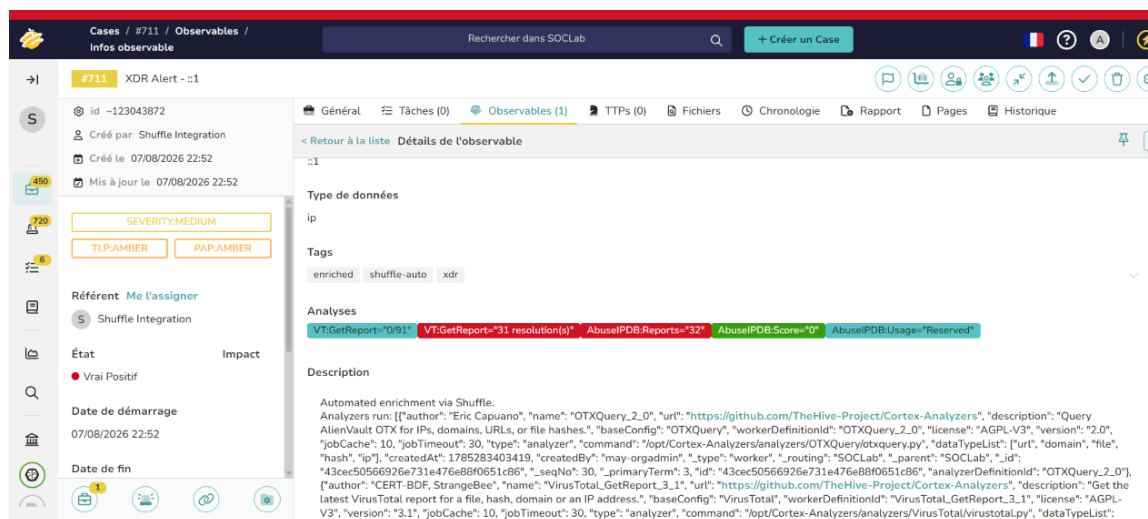


Figure 37: Automatically generated TheHive case #711 (“XDR Alert - ::1”) resulting from a brute-force test executed locally on the Wazuh host, hence the loopback address rather than a routable attacker IP; the enrichment analyses from VirusTotal and AbuseIPDB are otherwise unaffected.

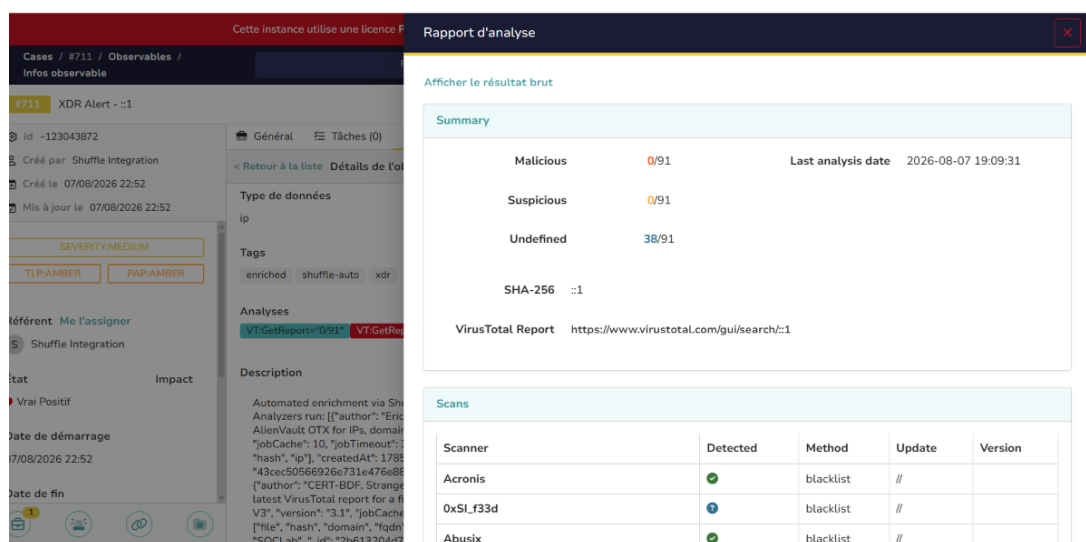


Figure 38: VirusTotal analyzer report for the same observable: 0 malicious / 0 suspicious detections out of 91 engines, with 38 engines returning an undefined verdict.

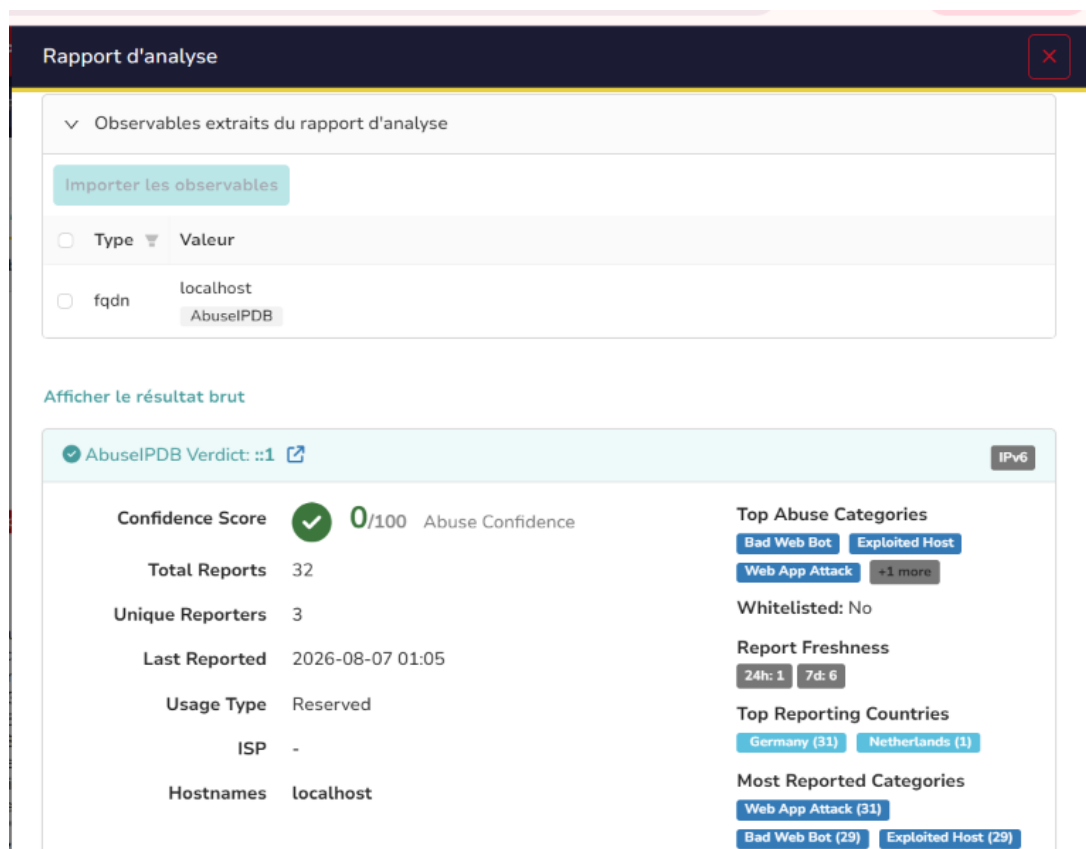


Figure 39: AbuseIPDB analyzer report attached to the brute-force case: 32 total abuse reports, categorized as Web App Attack, Bad Web Bot, and Exploited Host.

11.3 Scenario 2: Cowrie Honeypot Interaction

A second scenario validated the honeypot-specific detection path: a simulated attacker authenticates successfully against Cowrie, issues a command, and disconnects – generating three distinct Wazuh rules (100100, 100102, 100103) in sequence.

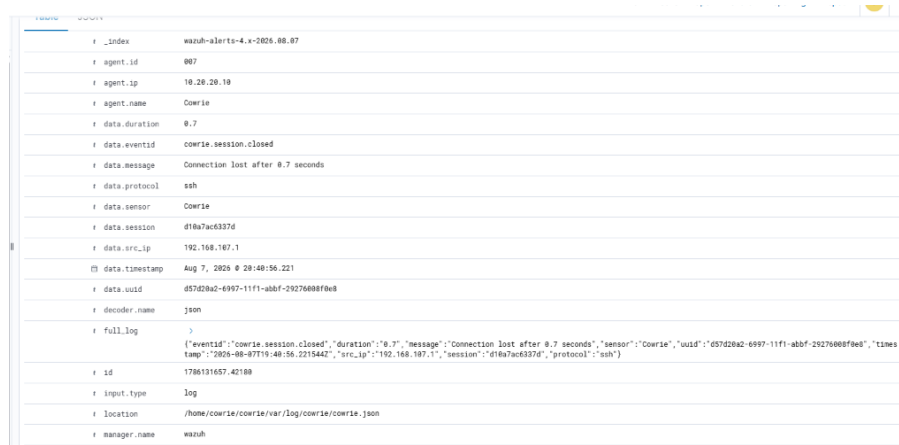


Figure 40: Wazuh rule detail for the Cowrie rule family (100100–100103), covering successful/-failed logins, command execution, and session closure.

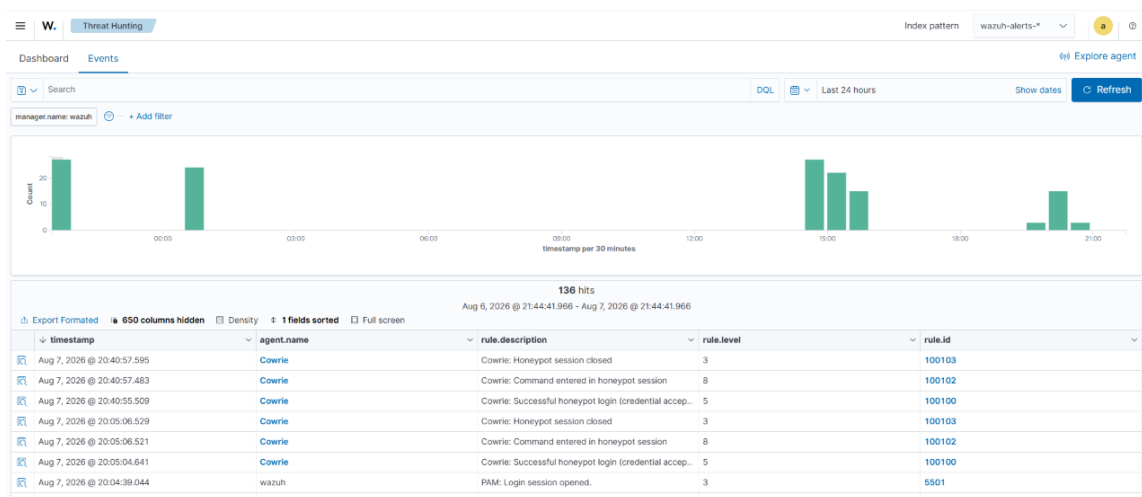


Figure 41: Discover view of raw Cowrie session events ingested into Wazuh: session close, command input (`exit`), and successful login, each carrying the full JSON payload used downstream by the Active Response script.

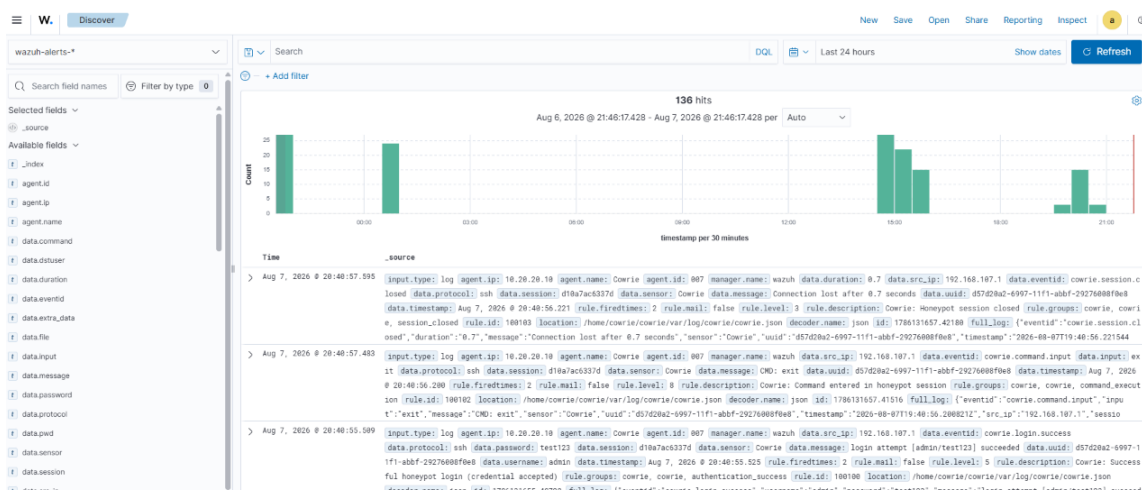


Figure 42: Threat Hunting timeline of Cowrie honeypot events over a 24-hour window, showing the characteristic burst pattern of automated/simulated login attempts (136 hits).

12 Threat Hunting and Monitoring Dashboards

Beyond the automated response pipeline, the Wazuh dashboard provides a consolidated threat-hunting view across all monitored assets, useful for manual triage and trend analysis.

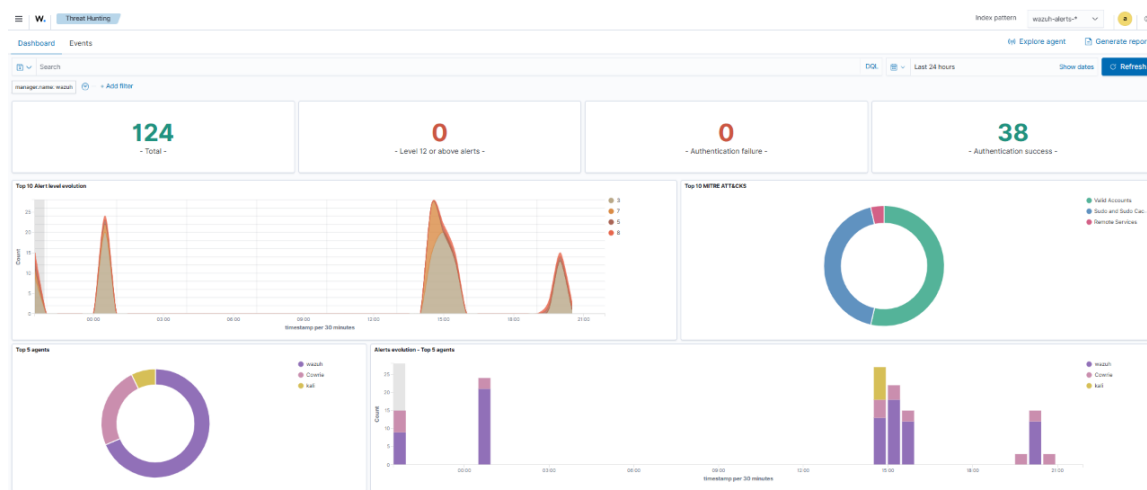


Figure 43: Wazuh Threat Hunting dashboard (last 24 hours): 124 total alerts, 38 authentication successes, MITRE ATT&CK technique breakdown (Valid Accounts, Sudo and Sudo Caching, Remote Services), and top-contributing agents (wazuh, Cowrie, kali).

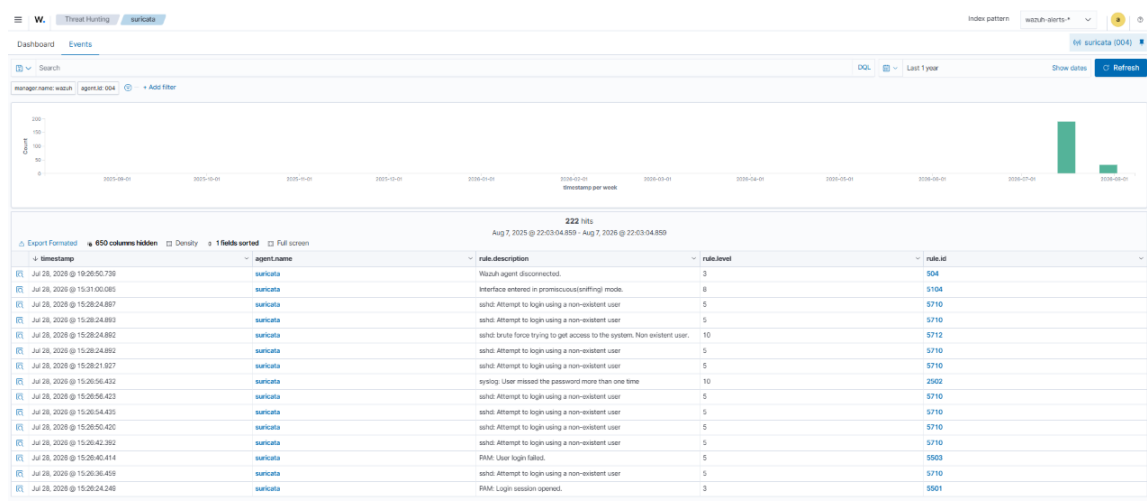


Figure 44: Suricata-sourced Wazuh events over the past year (222 hits), dominated by SSH brute-force and non-existent-user login attempts.

13 Challenges and Lessons Learned

- Active Response dispatch semantics.** The `<location>local</location>` vs. `<location>server</location>` distinction is easy to overlook but critical when the alert originates from an agent (like Cowrie) rather than the manager. This was the single largest source of “silent failure” during development – the response appeared configured correctly but never executed.
- Debugging with minimal tooling.** Diagnosing Active Response issues required a disciplined loop of truncating `active-responses.log`, manually reproducing the trigger condition, and inspecting raw `curl` verbose output against the Shuffle webhook – there is no high-level UI for this layer of the stack.
- Webhook URL churn.** Shuffle regenerates webhook UUIDs when a workflow is duplicated or re-saved in certain conditions, which silently broke the pipeline more than once until the URL was pinned and re-verified after every workflow edit.

- **Designing for auditability, not just automation.** Even though the entire pipeline runs unattended, TheHive’s task templates (Triage, Enrichment Review, Containment Decision) ensure every automated action leaves a reviewable trail, which matters for governance-minded environments like the one I support during my SAA internship.

14 Conclusion and Future Work

This project delivered a working, end-to-end automated detection-to-containment pipeline built entirely from open-source components in a home lab setting. The migration away from Splunk simplified operations while adding genuine SOAR capability that the previous setup lacked. Beyond the technical build, the debugging process around Wazuh’s Active Response dispatch model deepened my understanding of how SIEM and SOAR platforms actually communicate at the protocol level, rather than treating them as black boxes.

Planned next steps include:

- .
- Adding MITRE ATT&CK-mapped detection rules for lateral movement techniques (Kerberoasting, Pass-the-Hash) previously simulated in the Active Directory portion of the lab, feeding the same automated pipeline.
- Publishing the full Shuffle workflow JSON and Wazuh custom rules to the public GitHub portfolio for reproducibility.

Full source, workflow exports, and detection rules available at:
<https://github.com/mayversion/soc-analyst-portfolio>